

DriftShield AI

*Lightweight, Local-First Production Model Monitoring Engine
with Validated Statistical Drift Detection*

Project Documentation & Build Plan

A small, honest monitoring tool that proves its alerts mean something before asking anyone to trust them.

1. Overview

DriftShield AI is an open-source, lightweight monitoring tool for tracking the health, behavior, and data distributions of machine learning models in production. It is built local-first: developers run it next to their own inference pipeline, store telemetry in a local SQLite database, and get drift alerts without sending data to an external service or paying for cloud infrastructure.

Most monitoring tools fall into one of two traps: they wrap a single statistical test (e.g. Kolmogorov-Smirnov) around incoming data and call it “drift detection” without ever checking whether the test actually fires correctly, or they bring enterprise-scale complexity (Kubernetes, managed cloud services) that a solo developer or small team doesn't need. DriftShield AI is deliberately positioned between these: it does real statistical validation, but it is small enough to read end-to-end and run on a laptop.

Core design principle: a monitoring tool is only as trustworthy as its false alarm rate.

A drift detector that fires constantly on ordinary statistical noise will be ignored within a week by any team that adopts it. The core engineering effort of DriftShield AI goes into proving — not assuming — that its alerts mean something, using a validation harness built directly into the project rather than left as a manual exercise.

2. Problem Statement

Once a model is deployed, its accuracy on live traffic is invisible by default, because ground-truth labels usually arrive late or never. Two kinds of drift can occur silently in the meantime:

- **Covariate drift** — the distribution of input features changes (e.g. a recommender starts seeing a new customer segment), even if the relationship between inputs and the correct output hasn't changed.
- **Concept drift** — the relationship between inputs and outputs changes (e.g. customer behavior shifts after a macroeconomic event), even if the input distribution looks the same.

These require different responses: covariate drift may not need retraining at all if the model generalizes well outside its training distribution; concept drift usually does need retraining or intervention, because the model's learned mapping is now wrong. A monitoring tool that doesn't distinguish between them gives an incomplete picture and can lead to the wrong decision.

A second, more practical problem is statistical: if a system runs the same significance test independently across many features, it will produce false positives by chance alone at a rate far higher than the nominal significance level suggests. Without correcting for this, a “monitoring dashboard” becomes a noise generator.

3. Design Goals

The project is scoped around four goals, in priority order:

1. **Validated detection, not assumed detection.** Every drift-detection method shipped in this project must be evaluated against a controlled scenario (known drift vs. known non-drift) before being trusted in the dashboard, with results published as part of the project itself.
2. **Local-first and dependency-light.** No required cloud services, no recurring cost, no external data exposure. SQLite for storage, FastAPI for ingestion, Streamlit for visualization.
3. **Honest about uncertainty.** The system reports confidence and false-positive risk alongside any alert, not just a binary “drift detected” flag.
4. **Small enough to defend line-by-line.** Every statistical or architectural decision should be explainable in a technical interview, including its limitations.

4. System Architecture

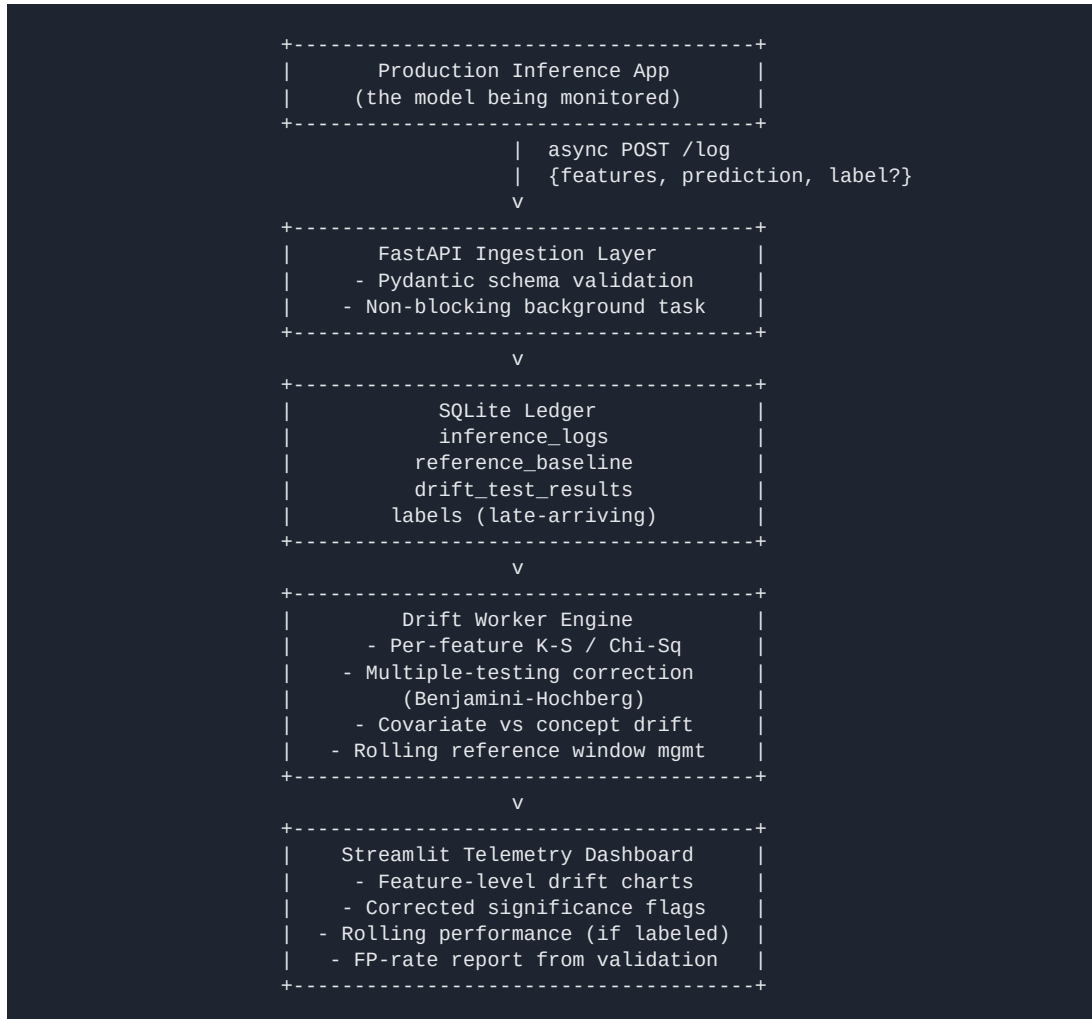


Figure 1. End-to-end data flow from live inference through ingestion, storage, drift detection, and visualization.

A fifth, parallel module — the **Validation Harness** — does not sit in the live request path. It is a separate, offline component that runs the drift detection logic against synthetic and real benchmark scenarios where the “true” drift status is known in advance, and produces the false-positive and true-positive numbers referenced throughout this document. It is what separates this project from a system that merely runs a statistical test.

5. Core Modules

5.1 Ingestion Layer (main.py)

A FastAPI service exposing a single primary endpoint, `POST /Log`, that accepts a feature vector, a model prediction, and an optional ground-truth label (for cases where labels arrive immediately or are backfilled later). Logging is dispatched to a background task so it never blocks the calling application's inference latency. Input is validated with Pydantic so malformed payloads fail fast and visibly rather than corrupting the telemetry store.

5.2 Storage Layer (database.py)

SQLite is used deliberately rather than a heavier database, in keeping with the local-first goal. Four tables: `inference_logs` (every prediction event), `reference_baseline` (a snapshot of the training/reference distribution per feature), `drift_test_results` (every statistical test run, its raw p-value, its corrected significance flag, and which drift type it addresses), and `labels` (ground truth, joined back to `inference_logs` by request ID when available).

5.3 Drift Detection Engine (drift_engine.py)

This is the core of the project.

For each numerical feature, the engine runs a two-sample Kolmogorov-Smirnov test comparing a rolling production window against the reference baseline; for categorical features, a Chi-Square test of independence. Critically, the engine does not report raw p-values as final verdicts. Because tests run across many features on a rolling schedule, the engine applies a Benjamini-Hochberg false discovery rate correction across the batch of simultaneous tests before flagging anything as significant. This single decision is the difference between a system that's honest about multiple comparisons and one that fires constantly on noise.

Where a label is available (even delayed), the engine separately tracks **concept drift** by monitoring the model's live performance metric (e.g. F1, calibration error) against its known training-time performance, distinct from the **covariate drift** signal coming from the feature-distribution tests. The dashboard always shows these as two separate signals, never collapsed into one “drift score,” because they imply different actions.

The reference baseline is not fixed forever: the engine supports an explicit, user-controlled reference window policy (static-from-training, or rolling-N-days), and the documentation states plainly that choosing “rolling” trades the ability to detect slow drift for a lower false-alarm rate, since a rolling window partly absorbs gradual change into the new baseline.

5.4 GenAI Output Evaluation (llm_eval.py) — scoped deliberately small

For teams monitoring an LLM-based system rather than a tabular model, this module uses an LLM-as-judge pattern (via Groq or Gemini's fast inference APIs) to flag likely hallucination or format drift in generated text. This module is explicitly labeled in the dashboard and documentation as **heuristic and**

unvalidated against human judgment by default — it ships with an optional calibration step where a developer can supply a small set of human-labeled examples (e.g. 50–100 outputs labeled “hallucinated” / “not hallucinated”) and the system reports the judge's agreement rate against those labels before anyone relies on it. Without that calibration step run, the dashboard displays a visible warning rather than presenting the LLM-judge output as a trustworthy signal. This module is intentionally not the centerpiece of the project.

5.5 Validation Harness (validation/)

This module is what makes the rest of the project defensible, and is treated as a first-class deliverable rather than a test suite.

It works as follows:

1. Take a real public dataset (e.g. a tabular classification dataset with a clear time or batch structure).
2. Construct a “no drift” scenario: split the data into reference and production windows drawn from the same distribution, and run the full detection pipeline. Record the false positive rate — how often the system raises an alert when nothing actually changed.
3. Construct several “known drift” scenarios with different drift types and magnitudes: synthetic covariate shift (resampling a feature's distribution), synthetic concept drift (flipping the relationship between a feature and the label for a subset of data), and a real-world drift scenario if the dataset supports it. Run the full pipeline against each and record the true positive rate and detection delay (how many batches until the system flags it).
4. Publish all of these numbers — not just the passing cases — in the project's results documentation, including cases where detection was slow or missed, with an explanation of why.

The headline claim of the project is not “DriftShield AI detects drift.” It is a specific, falsifiable claim of the form:

“On dataset X, DriftShield AI's corrected K-S test detected synthetic covariate shift of magnitude Y within Z batches, with a false positive rate of W% on non-drifted control data.”

That sentence is the actual deliverable.

6. Repository Structure

```
driftshield-ai/
|-- main.py           # FastAPI ingestion gateway
|-- database.py       # SQLite schema + connection handling
|-- drift_engine.py   # Drift core (K-S, Chi-Sq, BH correction)
|-- llm_eval.py       # GenAI eval (calibration-gated, optional)
|-- dashboard.py      # Streamlit telemetry UI
|-- validation/
|   |-- run_validation.py # Orchestrates all benchmark scenarios
|   |-- scenarios.py      # No-drift / synthetic / real-drift cases
|   |-- results/          # Published FP/TP rates, versioned
|-- tests/
|   |-- test_drift_engine.py # Unit tests, statistical functions
|-- requirements.txt
`-- README.md          # Leads with the validated headline claim
```

7. What This Project Demonstrates (and What It Doesn't)

Demonstrates

- Statistical literacy applied correctly under realistic constraints — multiple comparisons, distinguishing drift types, defining and measuring false-positive rate.
- Systems thinking about production ML — async ingestion, label latency, reference window tradeoffs.
- Honest scoping of an LLM-based feature rather than overclaiming it.
- The discipline to validate a tool's own claims rather than asserting them.

Does Not Demonstrate

- Novel statistical methods — K-S and Chi-Square are standard, well-known tests. The contribution here is correct, validated application, not invention.
- Large-scale distributed systems engineering — this is intentionally a single-node, local-first tool, not a Kubernetes-scale platform.

The documentation states this explicitly rather than implying a broader scope than what was built.

8. Build Roadmap

Phase	Deliverable
1	SQLite schema, FastAPI ingestion endpoint, basic Streamlit dashboard skeleton (no real detection logic yet)

2	K-S and Chi-Square detection implemented per feature, with raw p-values surfaced
3	Benjamini-Hochberg correction across simultaneous tests; concept vs. covariate drift separated in storage and dashboard
4	Validation harness built against a real public dataset; no-drift and synthetic-drift scenarios run; false-positive and true-positive rates published in validation/results/
5	Reference window policy (static vs. rolling) made configurable, with tradeoffs documented
6	Optional GenAI evaluation module added, gated behind the calibration-check warning
7	Unit tests for all statistical functions in tests/; final README rewritten to lead with the validated headline claim from Phase 4

Phase 4 is the non-negotiable core of the project. Everything before it is plumbing; everything after it is extension.

DriftShield AI — a small, honest monitoring tool that proves its alerts mean something before asking anyone to trust them.